Control-Plane Guided Autonomous Research System

Stage 1 Prototype — Harness-Mediated Autoresearch Runtime

April 2026 · Internal Technical Review

Executive Summary

This report presents a harness-mediated runtime for autonomous experimentation: a governed loop in which a strong planning model directs a bounded local agent through an explicit control plane, producing auditable, reproducible research progress without manual iteration.

A full 3 × 5 campaign (15 governed runs) on a ResNet-based binary classifier improved validation AUC from 0.7799 → 0.7876 (+0.77%) autonomously, with every decision logged, every code change git-tracked, and zero data leaving the machine.

Why this matters

This system directly addresses a key bottleneck in scalable research workflows: turning experimentation into a controlled, auditable, and repeatable process. The architecture is designed for enterprise requirements — local/private execution, multi-node scaling, and model-agnostic orchestration — and aligns with building a practical research operating system rather than a single agent.

Core properties:

- Local / private execution — no data leaves the machine (enterprise IP requirement)
- Model-agnostic orchestration — any LLM backend can plug into the same harness
- Multi-node scaling — the control plane API is node-agnostic by design

Campaign Results

The 3 × 5 campaign ran three manager rounds of five worker experiments each. The manager received only a structured status and memory summary between rounds — no model weights, no raw data.

Metric	Value	Notes
Baseline validation AUC	**0.7799**	Starting point, no iteration
Best validation AUC	**0.7876**	R3W4: LR=5e-4, WD=3e-5, dropout=0.02
Improvement	**+0.0077 (+0.77%)**	Fully autonomous

Experiments run / kept	15 run / 5 kept	All keep/discard decisions logged with rationale
Execution	100% local CPU	Ollama + Qwen 2.5 7B Coder — no external API

The campaign demonstrated emergent explore-then-exploit behavior: Round 1 screened learning rates, Round 2 explored architectural depth (largely unsuccessful), Round 3 exploited fine-grained regularization around the best-found configuration. This strategy emerged from the manager's memory-guided reasoning — it was not programmed explicitly.

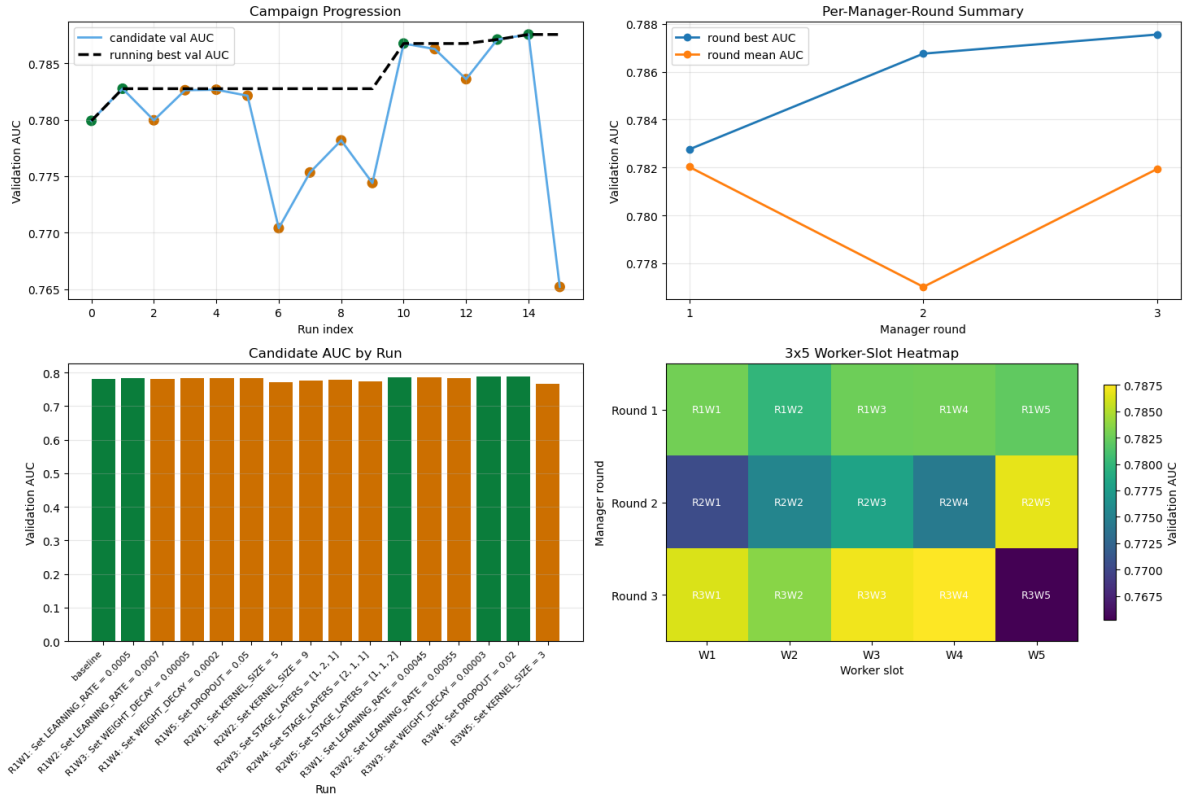

Figure 1. Campaign progression: candidate AUC per run (top-left), per-round summary (top-right), per-run bar chart with keep/discard coloring (bottom-left), worker-slot heatmap (bottom-right).

System Architecture

Control Plane (Governance Layer)

The control plane functions as a harness-style runtime — inspired by open-claw / agentic execution frameworks — mediating all agent actions through an explicit execution layer.

This decouples reasoning (manager), execution (worker), and governance (control plane), and allows different models to plug into the same controlled experimentation system.

It owns the experiment state machine, enforces the lifecycle contract (no run without an explicit `/run` request, no memory commit without a rationale), and maintains an append-only JSONL log. The pending-run guard prevents entangled experiments: any new run request is rejected while a prior experiment is still active.

Manager (Planning Layer)

The manager reads `/status` and `/memory-summary` before each round, then proposes a bounded objective — one or two hyperparameter changes to `train.py` — and issues `/keep` or `/discard` with an explicit rationale. Stage 1 used an interactive Codex session in `manual_codex` mode, enabling step-by-step inspection before full `/loop` automation.

Worker (Execution Layer)

A bounded code-editing agent constrained to a single file (`train.py`). A hard assertion verifies file-level scope at commit time:

```
assert changed_files == ["train.py"]
```

This makes every performance change unambiguously attributable to a specific code diff. No other file is touched; the experiment history is therefore trustworthy by construction.

Key Design Properties

Property	What it means in practice
Auditability	Every experiment: git commit hash + metric + keep/discard + rationale. Fully reproducible by commit checkout.
Data privacy	Worker model runs fully local (Ollama). No training data, weights, or intermediate results leave the machine.
Memory-guided iteration	Manager reads structured memory before each round. Prevents redundant experiments; enables adaptive strategy.
Measurable decisions	Keep/discard criterion: delta = candidate_auc − best_auc > 0. No heuristics, no opaque reasoning.
Model-agnostic	Manager and worker are swappable. Any OpenAI-compatible backend plugs into the same control plane.

Comparison to Traditional Workflow

Dimension	Manual	This System
Pace	1–2 experiments / day	15 governed runs overnight

Memory	Researcher notes; siloed	Append-only JSONL; persistent
Reproducibility	Ad-hoc git discipline	Validated diff + commit hash per run
Decision accountability	Tacit knowledge	Explicit rationale stored per experiment
Data governance	Cloud tool leakage risk	100% local — no external API

Roadmap

Stage 2 — Automated Loop: Replace manual_codex with full /loop execution. Target: 5-round × 10-worker overnight campaign to verify multi-round strategy convergence.

Stage 3 — Multi-Node / Parallel Branches: The control plane API is node-agnostic. Directing a second worker node requires only a new node root. Parallel branch managers can then race hypotheses and merge results.

Stage 4 — Domain Integration: Optimal filter tuning for near-threshold trigger logic, EMPCA basis optimization, and threshold parameter sweeps. The prepare.py/train.py separation provides a natural safety boundary for proprietary data pipelines.

Optional — Dashboard: Lightweight React UI for campaign visualization, memory summaries, and live experiment status. Ready for implementation; supports broader team adoption.

Conclusion

The Stage 1 prototype delivers a +0.77% AUC improvement from a fully autonomous 15-run campaign, with a complete, inspectable record of every decision. The emergent explore-then-exploit behavior — breadth search in Round 1, architectural probing in Round 2, fine-grained exploitation in Round 3 — arose from the manager's memory-guided reasoning without explicit programming of that strategy.

The key contribution is not the specific model improvement, but the demonstration that a harness-mediated system can produce reliable, auditable research progress under strict execution constraints. This is the foundation of a practical research operating system — one that extends directly to multi-node orchestration, private enterprise data pipelines, and model-agnostic experimentation.